

Set 18: Using Dictionaries

Skill 18.1: Access key values

Skill 18.2: Validate a dictionary key

Skill 18.3: Use the *get* method to retrieve a key from a dictionary

Skill 18.4: Use the *pop* method to delete a key from a dictionary

Skill 18.5: Retrieve dictionary keys

Skill 18.6: Retrieve dictionary values

Skill 18.7: Retrieve key:value pairs from dictionaries

Skill 18.1: Access key values

Skill 18.1 Concepts

Once you have a dictionary, you can access the values in it by providing the key. For example, let's imagine we have a dictionary that maps buildings to their heights, in meters. Below illustrates how we can access the values associated with different buildings.

Code
<pre>building_heights = {"Burj Khalifa": 828, "Shanghai Tower": 632, "Abraj Al Bait": 601, "Ping An": 599, "Lotte World Tower": 554.5, "One World Trade": 541.3} print(building_heights["Burj Khalifa"]) print(building_heights["Ping An"])</pre>
Output
828 599

Skill 18.1 Exercise 1

Skill 18.2: Validate a dictionary key

Skill 18.2 Concepts

Let's say we have our dictionary of building heights from the last exercise, and we wanted to know the height of the Landmark 81 in Ho Chi Minh City?

Code
<pre>building_heights = {"Burj Khalifa": 828, "Shanghai Tower": 632, "Abraj Al Bait": 601, "Ping An": 599, "Lotte World Tower": 554.5, "One World Trade": 541.3} print(building_heights["Landmark 81"])</pre>
Output
KeyError: 'Landmark 81'

Because *Landmark 81* does not exist in the dictionary, we get a *key error*.

One way to avoid this error is to first check if the key exists in the dictionary,

Code
<pre>building_heights = {"Burj Khalifa": 828, "Shanghai Tower": 632, "Abraj Al Bait": 601, "Ping An": 599, "Lotte World Tower": 554.5, "One World Trade": 541.3} key_to_check = "Landmark 81" if key_to_check in building_heights: print(building_heights["Landmark 81"]) else: print("not in dictionary")</pre>
Output
not in dictionary

The code above will not throw an error, because *key_to_check in building_heights* will return *False*, and so we never try to access the key.

[Skill 18.2 Exercise 1](#)

Skill 18.3: Use the *get* method to retrieve a key from a dictionary

Skill 18.3 Concepts

We saw in the last exercise that we had to add a key:value pair to a dictionary in order to avoid a *KeyError*. This solution is not sustainable. We can't predict every key a user may call and add all of those placeholder values to our dictionary.

Dictionaries have a *get()* method to search for a value instead of the *my_dict[key]* notation we have been using. If the key you are trying to *get()* does not exist, it will return *None* by default:

Code
<pre>building_heights = {"Burj Khalifa": 828, "Shanghai Tower": 632, "Abraj Al Bait": 601, "Ping An": 599, "Lotte World Tower": 554.5, "One World Trade": 541.3} key_to_check = "Landmark 81" print(building_heights.get("Shanghai Tower")) print(building_heights.get("My House"))</pre>
Output
632 none

You can also specify a value to return if the key doesn't exist. For example, we might want to return a building height of 0 if our desired building is not in the dictionary,

Code
<pre> building_heights = {"Burj Khalifa": 828, "Shanghai Tower": 632, "Abraj Al Bait": 601, "Ping An": 599, "Lotte World Tower": 554.5, "One World Trade": 541.3} key_to_check = "Landmark 81" print(building_heights.get('Shanghai Tower', 0)) print(building_heights.get('Mt Olympus', 0)) print(building_heights.get('Kilimanjaro', 'No Value')) </pre>
Output
<pre> 632 0 No Value </pre>

[Skill 18.3 Exercises 1](#)

Skill 18.4: Use the *pop* method to delete a key from a dictionary

Skill 18.4 Concepts

Sometimes we want to get a key and remove it from the dictionary. Imagine we were running a raffle, and we have this dictionary mapping ticket numbers to prizes.

When we get a ticket number, we want to return the prize and also remove that pair from the dictionary, since the prize has been given away. We can use *pop()* to do this. Just like with *get()*, we can provide a default value to return if the key does not exist in the dictionary,

Code
<pre> raffle = {223842: "Teddy Bear", 872921: "Concert Tickets", 320291: "Gift Basket", 412123: "Necklace", 298787: "Pasta Maker"} print(raffle.pop(320291, "No Prize")) print(raffle) print(raffle.pop(100000, "No Prize")) print(raffle) print(raffle.pop(872921, "No Prize")) print(raffle) </pre>
Output
<pre> Gift Basket {223842: "Teddy Bear", 872921: "Concert Tickets", 412123: "Necklace", 298787: "Pasta Maker"} No Prize {223842: "Teddy Bear", 872921: "Concert Tickets", 412123: "Necklace", 298787: "Pasta Maker"} Concert Tickets {223842: "Teddy Bear", 412123: "Necklace", 298787: "Pasta Maker"} </pre>

Skill 18.4 Exercises 1 and 2

Skill 18.5: Retrieve dictionary keys

Skill 18.5 Concepts

Sometimes we want to operate on all of the keys in a dictionary. For example, if we have a dictionary of students in a math class and their grades. We can get a list of students without the grades using the built-in `list()` function.

Code
<pre>test_scores = {"Grace": [80, 72, 90], "Jeffrey": [88, 68, 81], "Sylvia": [80, 82, 84], "Pedro": [98, 96, 95], "Martin": [78, 80, 78], "Dina": [64, 60, 75]} print(list(test_scores))</pre>
Output
<pre>['Grace', 'Jeffrey', 'Sylvia', 'Pedro', 'Martin', 'Dina']</pre>

Dictionaries also have a `keys()` that returns a `dict_keys` object. A `dict_keys` object is a *view* object, which provides a look at the current state of the dictionary, without the user being able to modify anything. The `dict_keys` object returned by `keys()` is a set of the keys in the dictionary. You cannot add or remove elements from a `dict_keys` object, but it can be used in the place of a list for iteration,

Code
<pre>test_scores = {"Grace": [80, 72, 90], "Jeffrey": [88, 68, 81], "Sylvia": [80, 82, 84], "Pedro": [98, 96, 95], "Martin": [78, 80, 78], "Dina": [64, 60, 75]} for student in test_scores.keys(): print(student)</pre>
Output
<pre>Grace Jeffrey Sylvia Pedro Martin Dina</pre>

Skill 18.5 Exercise 1

Skill 18.6: Retrieve dictionary values

Skill 18.6 Concepts

Dictionaries have a `values()` method that returns a `dict_values` object (just like a `dict_keys` object but for values!) with all of the values in the dictionary.

Code
<pre>test_scores = {"Grace": [80, 72, 90], "Jeffrey": [88, 68, 81], "Sylvia": [80, 82, 84], "Pedro": [98, 96, 95], "Martin": [78, 80, 78], "Dina": [64, 60, 75]} print(test_scores.values())</pre>
Output
dict_values([[80, 72, 90], [88, 68, 81], [80, 82, 84], [98, 96, 95], [78, 80, 78], [64, 60, 75]])

Below illustrates how we can print the values associated with each key in the dictionary,

Code
<pre>test_scores = {"Grace": [80, 72, 90], "Jeffrey": [88, 68, 81], "Sylvia": [80, 82, 84], "Pedro": [98, 96, 95], "Martin": [78, 80, 78], "Dina": [64, 60, 75]} for score_list in test_scores.values(): print(score_list)</pre>
Output
<pre>[80, 72, 90] [88, 68, 81] [80, 82, 84] [98, 96, 95] [78, 80, 78] [64, 60, 75]</pre>

[Skill 18.6 Exercise 1](#)

Skill 18.7: Retrieve key:value pairs from dictionaries

Skill 18.7 Concepts

You can get both the keys and the values with the *items()* method. Like *keys()* and *values()*, it returns a *dict_list* object. Each element of the *dict_list* returned by *items()* is a *tuple* consisting of the key, value pairs,

Code
<pre>biggest_brands = {"Apple": 184, "Google": 141.7, "Microsoft": 80, "Coca-Cola": 69.7, "Amazon": 64.8} print(biggest_brands.items())</pre>
Output
dict_items([('Apple', 184), ('Google', 141.7), ('Microsoft', 80), ('Coca-Cola', 69.7), ('Amazon', 64.8)])

Below illustrates how you can iterate over a *dict_items* object return by the *items()* method,

Code
<pre>biggest_brands = {"Apple": 184, "Google": 141.7, "Microsoft": 80, "Coca-Cola": 69.7, "Amazon": 64.8} for company, value in biggest_brands.items(): print(company + " has a value of " + str(value) + " billion dollars. ")</pre>
Output
Apple has a value of 184 billion dollars. Google has a value of 141.7 billion dollars. Microsoft has a value of 80 billion dollars. Coca-Cola has a value of 69.7 billion dollars. Amazon has a value of 64.8 billion dollars.

Skill 18.7 Exercise 1